Clase 14 · Diagrama de Secuencia del Sistema (SSD)

Bloque UML Análisis · Modelación y Diseño de Sistemas 2026 · UNC-ESCMB-AUSI

Agenda de la clase

- Qué es un SSD y por qué es distinto de un Diagrama de Secuencia interno
- Relación del SSD con el Caso de Uso
- Notación UML: actor, sistema, lifeline, activaciones
- Tipos de mensajes: síncrono, asíncrono, retorno, create/destroy
- Fragmentos combinados: loop, alt, opt, par, critical, break
- Operaciones del sistema: cómo derivar la firma
- Metodología paso a paso (Larman)
- Caso ChangoMás · Realizar pago en caja
- Ejercicio en clase + autoevaluación

Recap del bloque UML Análisis

- Clase 11 · Modelado OO + Diagrama de Contexto (el sistema visto desde afuera)
- Clase 12 · Casos de Uso (qué puede hacer el sistema)
- Clase 13 · Modelo de Dominio (de qué habla el sistema)
- Clase 14 · SSD (cómo interactúa el actor con el sistema)
- Clase 15 · Diagrama de Estados (cómo evoluciona)
- Clase 16 · Validación del Triángulo de Larman (cierre del análisis)

¿Qué es un SSD?

System Sequence Diagram — Craig Larman, Applying UML and Patterns §10.

Un SSD muestra la secuencia temporal de eventos que ocurren entre los actores externos y el sistema, tratado como una CAJA NEGRA.

- Cada mensaje del actor hacia el sistema es una operación del sistema
- Una instancia concreta de flujo (un escenario, no todas las variantes)
- Análisis, no diseño — todavía no nos importa cómo se implementa
- Larman: "derivarlo es una disciplina, no una formalidad"

SSD vs. Diagrama de Secuencia interno

Mismo símbolo visual, granularidad distinta.

Aspecto	SSD (caja negra)	DS interno (objetos)
Participantes	Actor + Sistema (uno solo)	Múltiples objetos colaborando
Mensajes	Operaciones del negocio	Llamadas a métodos internos
Fase	Análisis	Diseño detallado
Cuándo se usa	Temprano	Después del modelo de dominio
Trazabilidad	Hacia el UC	Hacia el diagrama de clases

Relación del SSD con el Caso de Uso

El SSD formaliza el flujo básico del UC.

- UC especificado (fully-dressed, flujo numerado)
- Elegir UN escenario (flujo básico o extensión crítica)
- Identificar eventos actor↔sistema en cada paso
- Dibujar el SSD con esa secuencia
- Extraer las operaciones del sistema (firma + retorno)

Cada SSD = un caso de uso + un escenario.

Propósito del SSD

- Transformar texto en interfaz · descripciones narrativas → operaciones invocables
- Definir el contrato API del sistema · firma, parámetros, retorno
- Guía para el diseño detallado · el DS interno expande cada operación del SSD
- Detectar parámetros faltantes · al precisar la firma aparecen datos no explicitados
- Base para pruebas · cada operación = un endpoint que se puede testear

¿Cuándo dibujar un SSD?

Sí vale la pena

- ✓ UC crítico que se va a implementar pronto
- ✓ Hace falta extraer la API pública del sistema
- ✓ Hay interacciones complejas (loops, alternativas)
- ✓ Vas a preparar casos de prueba de integración
- ✓ El sistema colabora con otros sistemas externos

NO lo necesitás si…

- ✗ El UC es un CRUD trivial
- ✗ Todo el flujo cabe en 2 pasos
- ✗ Ya tenés BDD/Gherkin cubriendo la secuencia
- ✗ El UC aún no está estabilizado

Ubicación en el mapa UML

Esta clase: SSD — segunda mitad del bloque de Interacción.

Vista	Diagrama	Pregunta que responde
Contexto	Contexto	¿Quién está alrededor del sistema?
Interacción	Casos de Uso + SSD	¿Qué pide el actor y cómo responde el sistema?
Estructural	Modelo de Dominio, Clases	¿De qué cosas habla el sistema?
Comportamiento	Estados, Actividades	¿Cómo evolucionan internamente?

Elementos: actor y sistema

Actor

- Se coloca a la izquierda del diagrama
- Línea punteada vertical descendente = línea de vida (lifeline)
- Puede ser humano o sistema externo
- Si hay varios actores, cada uno tiene su lifeline

Sistema (caja negra)

- Rectángulo con etiqueta :NombreSistema (instancia anónima)
- El nombre va subrayado — convención UML
- Los dos puntos al inicio indican "instancia de"
- Es uno solo — el sistema se ve como un todo

Línea de vida (lifeline)

Representación del tiempo vertical descendente.

- Comienza en el rectángulo de cada participante
- Línea punteada vertical
- Termina con una X si la entidad se destruye
- El tiempo "corre" hacia abajo — flechas siempre van de arriba hacia abajo

Tip pedagógico: leer el SSD como un guión de teatro — actor habla, sistema responde, en orden cronológico.

Barras de activación

Rectángulos angostos sobre la lifeline = el participante está EJECUTANDO.

- Cada vez que recibe un mensaje síncrono, comienza una activación
- Al devolver el control (retorno), termina la activación
- Útil para ver anidamiento de llamadas
- En SSDs simples se omiten; en complejos aportan claridad

Mensaje síncrono

Flecha sólida con punta rellena. Dirección actor → sistema.

- Es el tipo de mensaje más común en SSDs
- Representa una llamada: el actor espera respuesta
- Sintaxis: nombre(arg1: Tipo1, arg2: Tipo2)
- Nombre = verbo en infinitivo o imperativo
- Parámetros en vocabulario del dominio, no tipos técnicos

✅ ingresarItem(sku: String, cantidad: int)

❌ insertIntoDatabase(...)

Mensaje asíncrono

Flecha sólida con punta abierta (no rellena).

- Fire-and-forget: se envía y no se bloquea
- Típico con colas, webhooks, servicios de mensajería
- También para eventos de dominio
- Si hay respuesta, llega después por otro canal

En SSDs de análisis la mayoría son síncronos. Los asíncronos aparecen con integraciones externas.

Mensaje de retorno

Flecha punteada con cabeza abierta. Sistema → actor.

- Se dibuja sólo cuando aporta información
- Incluir tipo / valor de retorno: saldo: Dinero
- Si la operación no retorna nada, se omite
- En SSDs pedagógicos mostrar el retorno ayuda a cerrar la interacción

Sintaxis recomendada: nombreVariable: TipoDominio

Ej: recibo: Comprobante, lista: List<Producto>

Auto-mensaje, create y destroy

Auto-mensaje · el sistema se envía un mensaje a sí mismo (ej: validar()). Usar cuando se quiere visibilizar una operación interna relevante.

Create · «create» en flecha punteada. La lifeline del nuevo objeto empieza donde recibe el create.

Destroy · «destroy» en flecha punteada. La lifeline termina con X en la posición de la destrucción.

En SSDs son uso ocasional — la mayor parte del modelo es actor↔sistema.

Operaciones del sistema · firma

Cada mensaje entrante al sistema define una operación.

Sintaxis general:

```
nombreOperacion(parametro1: Tipo, parametro2: Tipo, ...): TipoRetorno
```

- El nombre expresa la intención del actor, no la implementación
- Los tipos son del dominio (no int32, sino Cantidad)
- El retorno se especifica con el tipo del dominio (Recibo, Turno)

Cada operación del sistema se convierte luego en un endpoint o controller del backend.

Ejemplo de tabla de operaciones derivadas

Esta tabla es entregable: cada fila se convierte en endpoint y en test de integración.

Firma	Retorno	Propósito
iniciarVenta()	void	Crea una nueva venta
ingresarItem(sku: String, cant: int)	void	Agrega una línea a la venta
finalizarVenta()	Venta	Cierra la venta e informa total
pagar(monto: Dinero)	Recibo	Registra el pago y emite recibo
cancelarVenta(id: VentaID)	void	Anula una venta en curso

Fragmentos combinados · overview

Marcos para modelar lógica de control dentro del SSD.

Fragmento	Significado
loop	Repetición mientras se cumpla la guarda
alt	Alternativa: una de varias ramas según condición
opt	Bloque opcional (un solo "if")
par	Paralelismo: ramas concurrentes
break	Sale del fragmento si se cumple la condición
critical	Bloque indivisible (no interrumpible)

Fragmento loop

Repetir un bloque hasta cumplir la guarda.

- Etiqueta loop en la esquina superior izquierda del marco
- Condición entre corchetes: [mientras haya items]
- Todo lo que esté dentro se repite
- Variantes:
- loop(3) · cantidad fija
- loop(1..*) · rango
- Sin guarda explícita → terminar manualmente

Ejemplo típico: agregar items a una venta hasta cerrarla.

Fragmentos alt y opt

alt · alternativas (if / else)

```
alt [saldo ≥ monto]      → retirar(monto) ; ok: Recibo
    [saldo < monto]      → error: Insuficiente
```

opt · bloque opcional (solo if)

```
opt [cliente quiere mail]
    → enviarComprobante(email)
```

opt es equivalente a alt con una sola rama.

Fragmentos par y critical

par · paralelo

Las ramas se ejecutan concurrentemente. Útil cuando una operación dispara múltiples acciones simultáneas (mail + sms + log).

critical · indivisible

El bloque se ejecuta sin interrupción. Típico de transacciones monetarias: debitar() + acreditar() deben ser atómicos.

En SSDs de análisis aparecen poco — más comunes en SSDs de diseño con integraciones reales.

Notas y restricciones temporales

UML permite anotaciones para agregar información al diagrama:

- { restricción } · condición invariante en ese punto
- { t < 3s } · restricción temporal sobre una operación
- Nota en post-it · explicación libre
- «estereotipo» · clasificación UML
- * etiqueta · referencia a extensión del CU

Ejemplo: { tiempoRespuesta < 3s } sobre una operación crítica.

Metodología · del CU al SSD

5 pasos para derivar un SSD:

- Elegir escenario · flujo básico o variante relevante
- Identificar actores involucrados · primario y secundarios
- Marcar eventos actor↔sistema · cada paso donde el actor "habla" con el sistema
- Convertir eventos en mensajes · nombres verbales + parámetros del dominio
- Modelar variantes con fragmentos · loops, alternativas, opcionales

Resultado: el SSD + la tabla de operaciones del sistema.

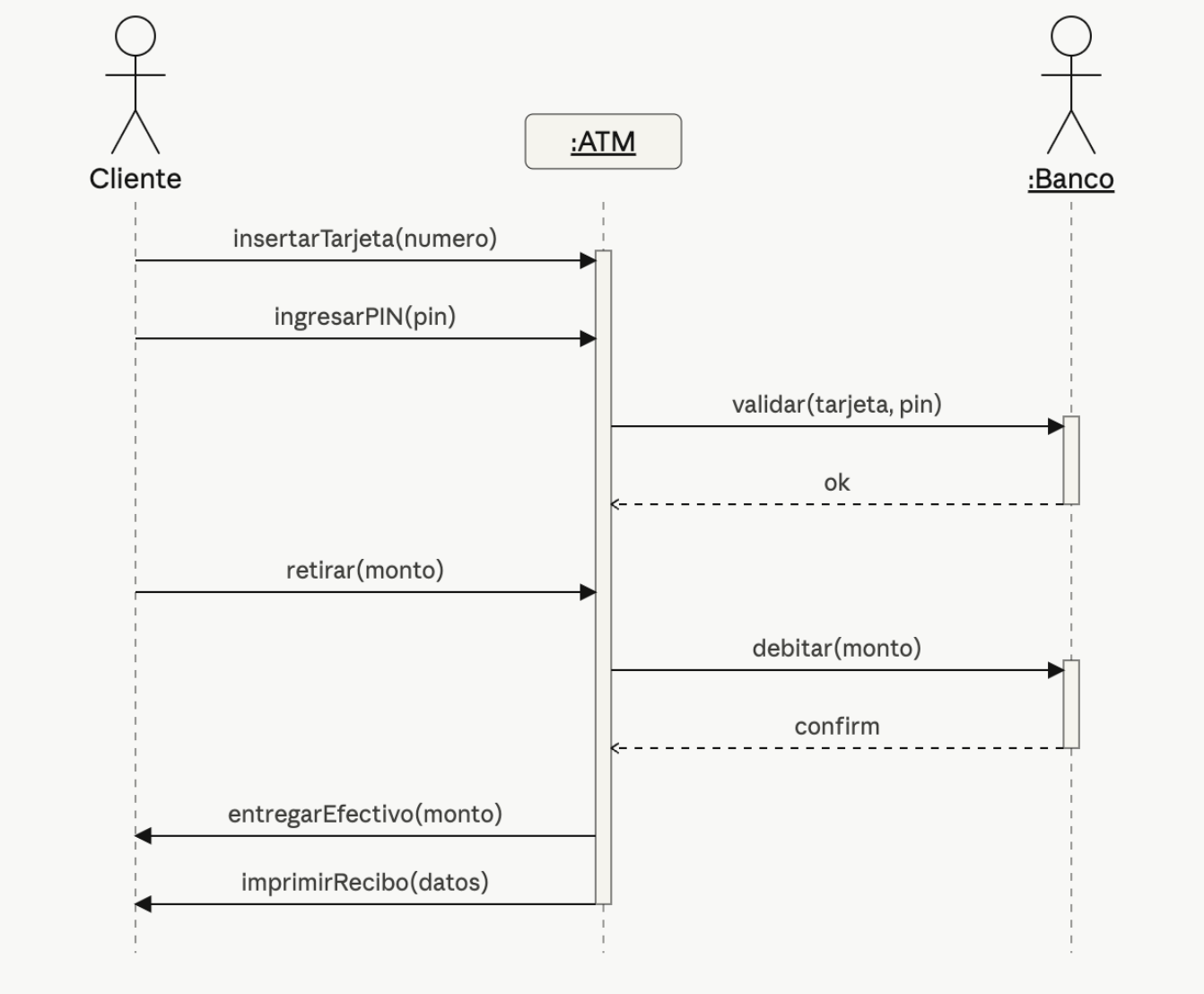
Convenciones y buenas prácticas

- Un SSD = un escenario · no mezclar flujos alternativos en el mismo diagrama
- Leer de arriba hacia abajo · el tiempo corre vertical
- Nombres en vocabulario de dominio · reservarTurno(), no insertIntoTurnosTable()
- Máximo 15-20 mensajes por SSD · si excede, partir en sub-escenarios
- Incluir tipos en parámetros · cantidad: int, precio: Dinero
- Mostrar el retorno solo si aporta · en SSDs de análisis suma claridad

Errores frecuentes (y cómo evitarlos)

- ❌ Modelar lógica interna del sistema — eso es DS interno, no SSD
- ❌ Omitir parámetros — pagar() sin monto es inútil
- ❌ Mensajes desde sistema al actor sin razón — el sistema sólo emite retornos, no inicia
- ❌ Mezclar múltiples UC en un diagrama — un SSD = un escenario
- ❌ Ignorar fragmentos cuando son necesarios — sin alt/loop el diagrama no captura variantes
- ❌ Nombres de UI — clickBotón, submitForm no van. El SSD está en nivel de negocio

Ejemplo 1 · ATM Retirar efectivo



Algunas observaciones sobre la notación:

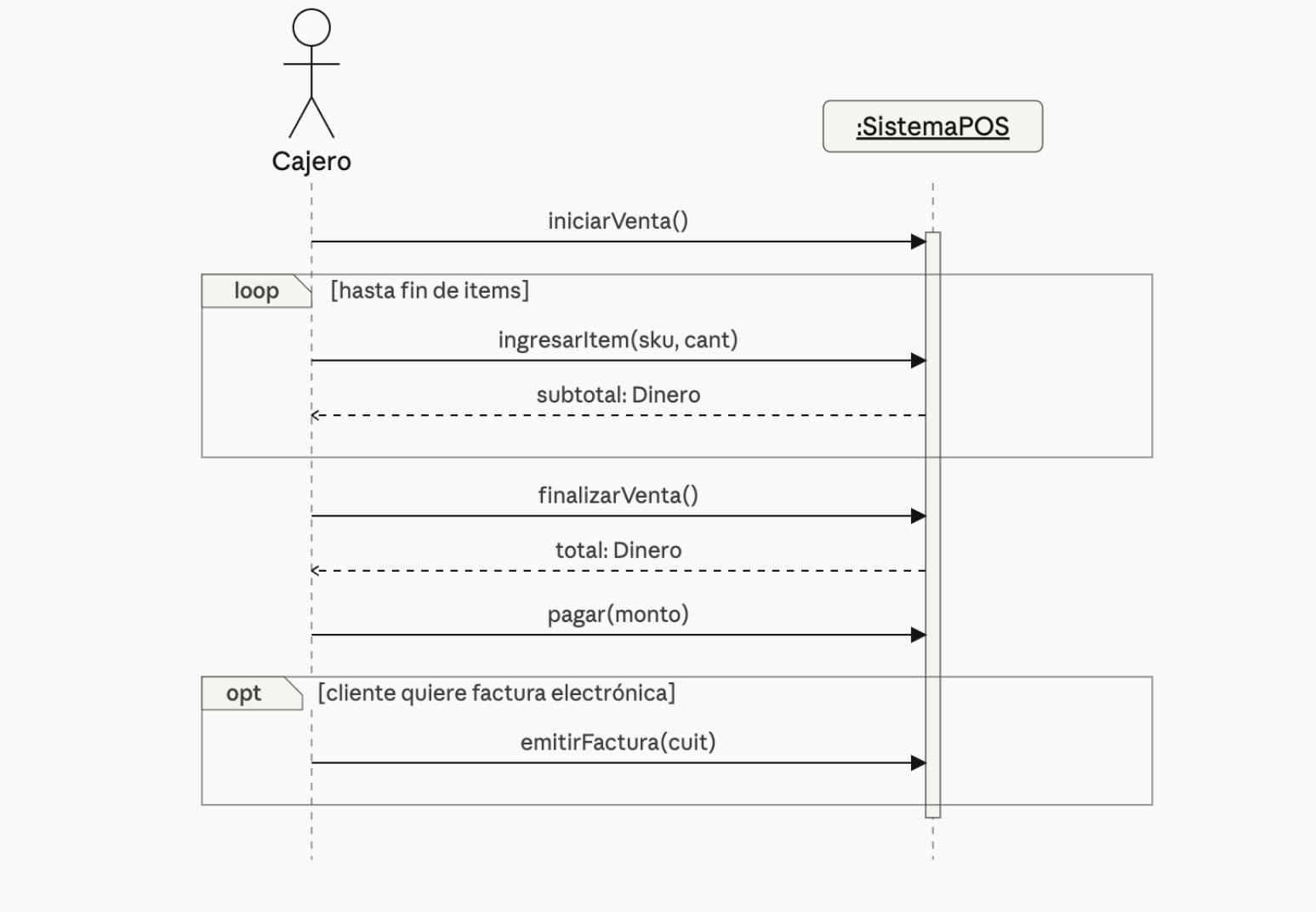
Cliente y :Banco dibujados como stick figures porque ambos son actores (primario y secundario respectivamente). El nombre con : y subrayado indica instancia anónima del rol.

:ATM como caja porque en un SSD el sistema se trata como caja negra.

Activación del :ATM abarca toda la operación; las del :Banco solo cubren las ventanas validar→ok y debitar→confirm, dejando ver que es un actor externo que solo está "activo" cuando se le invoca.

Mensajes síncronos con punta rellena (semántica de llamada con retorno esperado) y **retornos** con línea punteada + punta abierta (ok y confirm).

Ejemplo 2 · POS Procesar venta con loop + opt



Caso ChangoMás · CU "Realizar pago en caja de autoservicio"

Contexto: el cliente escanea productos en la caja de autoservicio, paga y se va con su ticket.

Actores:

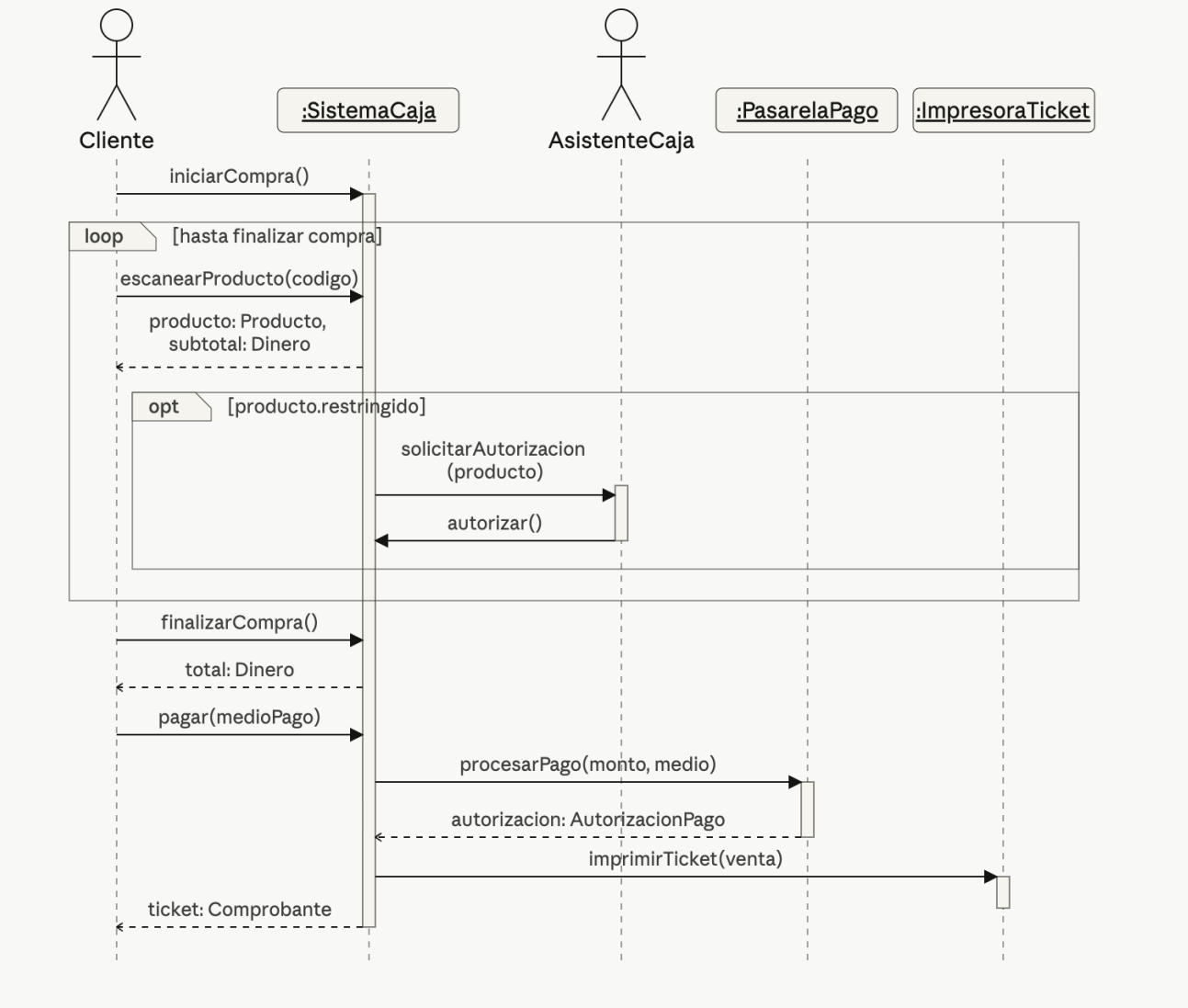
- Cliente (primario) — opera la caja
- AsistenteCaja (secundario) — interviene en productos restringidos
- PasarelaPago (secundario, sistema externo)
- ImpresoraTicket (secundario, dispositivo)

Pre-condición: la caja está libre y operativa.

ChangoMás · flujo básico del CU

- Cliente selecciona "Iniciar compra" en la caja
- Sistema crea una venta y muestra pantalla de escaneo
- Cliente escanea cada producto (puede ser varias veces)
- Sistema acumula el subtotal
- Si el producto es restringido (alcohol), sistema pide validación del Asistente
- Cliente selecciona "Pagar"
- Sistema solicita medio de pago a la Pasarela
- Pasarela autoriza el pago
- Sistema confirma la transacción
- Sistema imprime ticket

ChangoMás · SSD del flujo básico



Anidamiento de fragmentos — en ChangoMás el opt [producto.restringido] está físicamente *dentro* del rectángulo del loop [hasta finalizar compra]. Eso comunica que la autorización del asistente solo puede ocurrir por cada iteración del escaneo, no antes ni después.

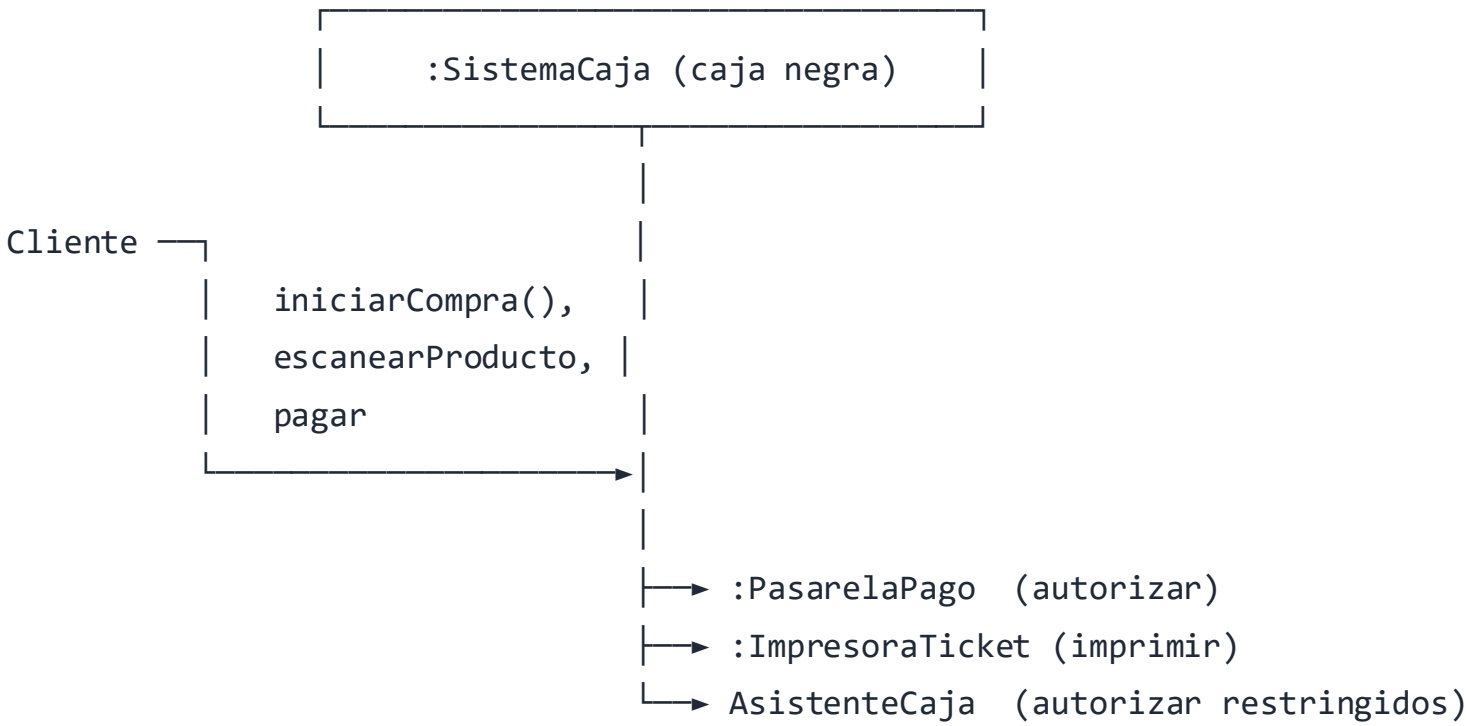
Activations cortas en actores secundarios — AsistenteCaja, :PasarelaPago y :ImpresoraTicket solo muestran barra de activación durante el lapso en que efectivamente "trabajan". Esto refuerza visualmente que son externos y que la actividad continua corresponde al :SistemaCaja, que es el sistema bajo estudio (SuD).

ChangoMás · operaciones del sistema derivadas

Estas 6 operaciones se convierten directamente en el contrato API del módulo de caja.

Firma	Retorno	Pre/post-condiciones
iniciarCompra()	Venta	Pre: caja libre. Post: venta EN_CURSO
escanearProducto(codigo: String)	LineaVenta	Pre: venta EN_CURSO. Post: línea agregada
solicitarAutorizacion(producto)	Solicitud	Solo si producto restringido
autorizar(solicitudId)	void	Pre: solicitud pendiente. Post: solicitud OK
finalizarCompra()	Dinero	Pre: ≥1 línea. Post: total calculado
pagar(medio: MedioPago)	AutorizacionPago	Pre: venta finalizada. Post: venta PAGADA

ChangoMás · diagrama mental



Actor primario opera; el sistema delega en secundarios cuando hace falta.

Trazabilidad UC ↔ SSD ↔ Operaciones

Un buen entregable de análisis tiene los tres alineados:

Si un paso del UC no tiene mensaje en el SSD → falta. Si un mensaje del SSD no tiene paso en el UC → sobra.

UC paso	Mensaje SSD	Operación derivada
1. Cliente selecciona "Iniciar"	iniciarCompra()	iniciarCompra(): Venta
3. Cliente escanea producto	escanearProducto(codigo)	escanearProducto(String): LineaVenta
5. Sistema pide validación	solicitarAutorizacion(producto)	solicitarAutorizacion(Producto): Solicitud
6. Cliente selecciona "Pagar"	pagar(medioPago)	pagar(MedioPago): AutorizacionPago

Ejercicio en clase · 20 minutos

Consigna:

Sobre el caso propio (OAS):

- Elegir un UC del trabajo de análisis (idealmente el crítico)
- Tomar el flujo básico numerado
- Identificar actores (primario y secundarios)
- Dibujar el SSD del flujo básico (papel o Draw.io)
- Usar al menos un fragmento combinado (loop, alt u opt)
- Derivar la tabla de operaciones del sistema con firma completa

Entregable: diagrama + tabla de operaciones en la carpeta del grupo.

Autoevaluación · Sistema de inscripción a materias

Enunciado

Un alumno ingresa al sistema con su legajo y contraseña. Consulta las materias disponibles del cuatrimestre, selecciona una o varias y confirma. El sistema valida correlativas contra el sistema académico, descuenta cupo y envía comprobante por email.

Consignas

- Dibujá el SSD del flujo básico
- Identificá los actores
- Usá al menos un fragmento (loop o alt)
- Derivá las operaciones del sistema con firma completa

Autoevaluación · solución

Actores: Alumno (primario), Sistema Académico y EmailSvc (secundarios)

Operaciones del sistema:

- login(legajo: String, password: String): Sesion
- consultarMaterias(cuat: String): List<Materia>
- seleccionar(materiaId: Long) · dentro de loop
- confirmarInscripcion(): Comprobante

Mensaje al Sistema Académico:

- validarCorrelativas(legajo, materiaId): Boolean

Mensaje asíncrono:

- enviarComprobante(email) (no espera respuesta)

Fragmentos: loop alrededor de seleccionar(), alt en confirmarInscripcion según resultado de validarCorrelativas.

Conexión con las próximas clases

- Clase 15 — Estados: cómo evoluciona internamente una Venta desde EN_CURSO → PAGADA → ANULADA
- Clase 16 — Validación del Triángulo de Larman: cierre del análisis, cruzar UC + Dominio + SSD + Estados
- Clase 17 — Clases de Diseño + SOLID: cómo cada operación del SSD se transforma en colaboraciones entre objetos internos

El SSD es la bisagra entre el análisis (lo que el sistema debe hacer) y el diseño (cómo lo va a hacer).

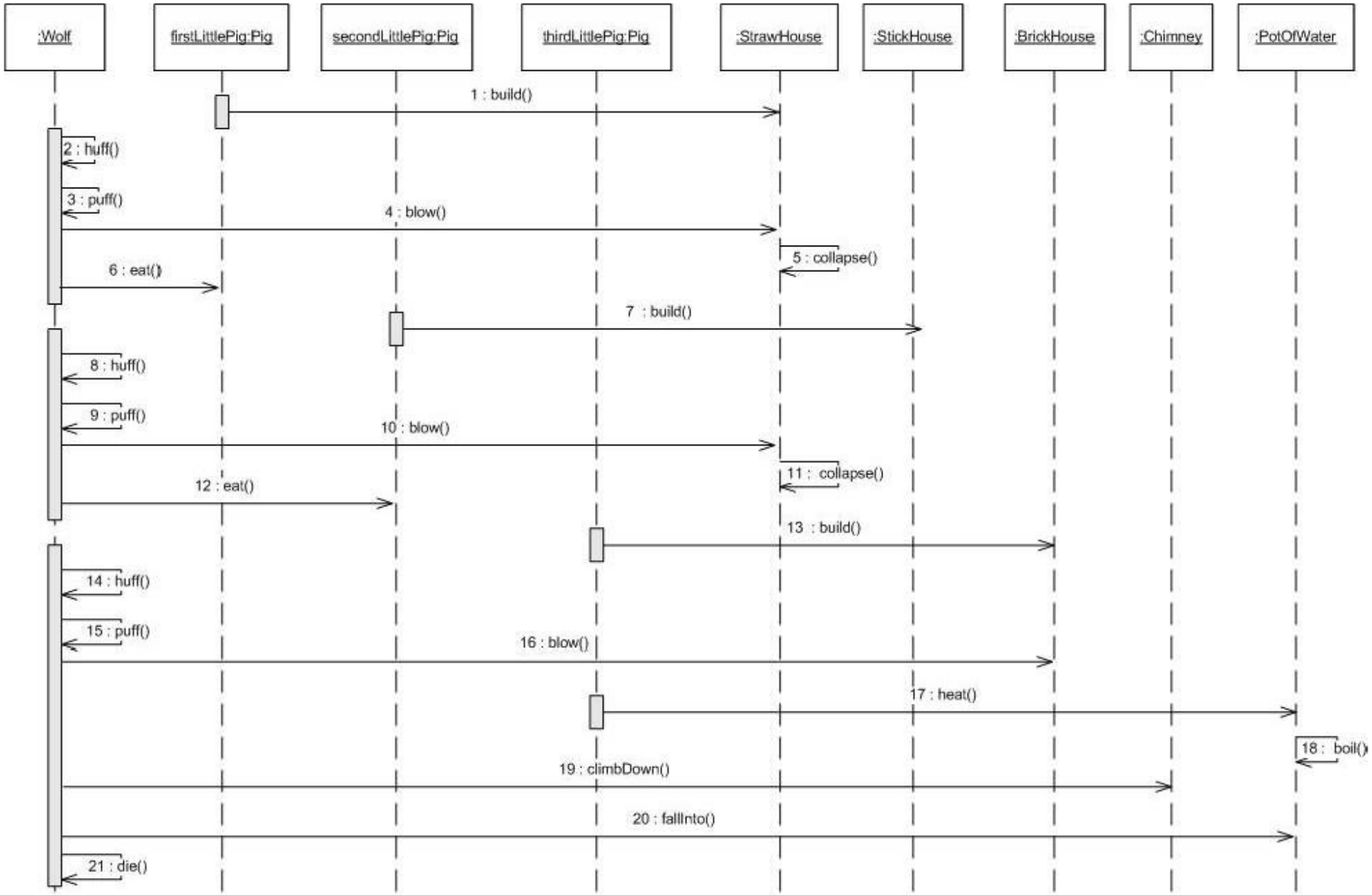
Tarea para casa

- Elegí 2 UC adicionales del caso propio y construí su SSD
- Para cada SSD, documentá la tabla de operaciones derivadas con pre/post-condiciones
- Validá con un compañero: que cada paso del UC tenga su mensaje y viceversa
- Subí los diagramas a la carpeta del grupo en ADO

Fecha: antes de Clase 15.



Ken Howard - fuente (discontinua) : http://kenhoward01.blogspot.com.ar/2008/06/three-little-pigs-in-uml.html


:Wolf
firstLittlePig:Pig
secondLittlePig:Pig
thirdLittlePig:Pig
:StrawHouse
:StickHouse
:BrickHouse
:Chimney
:PotOfWater
1 : build()
2 : huff()
3 : puff()
4 : blow()
5 : collapse()
6 : eat()
7 : build()
8 : huff()
9 : puff()
10 : blow()
11 : collapse()
12 : eat()
13 : build()
14 : huff()
15 : puff()
16 : blow()
17 : heat()
18 : boil()
19 : climbDown()
20 : fallInto()
21 : die()

Recursos

- Larman · Applying UML and Patterns (cap. 10 — System Sequence Diagrams)
- Fowler · UML Distilled (cap. Sequence Diagrams)
- OMG · UML 2.5.1 Specification §17 (Interactions)
- PlantUML — plantuml.com/sequence-diagram
- Draw.io — editor online gratuito

Próxima clase · Clase 15

Diagrama de Estados — el ciclo de vida interno de las entidades clave del dominio (Venta, Pedido, Turno).

Con el SSD aprendimos cómo el actor habla con el sistema en el tiempo. Ahora vamos a ver cómo evolucionan internamente las entidades cuando reciben esos mensajes.

Cierre · Preguntas

- ¿Para qué UC armarían un SSD y para cuál no?
- ¿Qué pasa si un SSD tiene más de 20 mensajes?
- ¿Cómo distinguen un actor primario de uno secundario?
- ¿Qué información agrega un SSD que el UC ya no tenga?

Dudas y consultas.